Concrete Architecture Report for Gemini CLI

Group 5: It Compiles (Sometimes)

Assignment 2

Friday, March 13, 2026

Group Leader: Edward Tanurkov - edward.tanurkov@queensu.ca

Presenter 1: Brian Constandes - 22bac1@queensu.ca

Presenter 2: Arham Hassan - 23cr8@queensu.ca

Ajwaad Khan - 19aak19@queensu.ca

Junoh Loy - 22DLT1@queensu.ca

Faizan Muhammad - 22mq34@queensu.ca

Table of Contents

1 Abstract

2 Introduction

3 Updated Conceptual Architecture

3.1 Revised Subsystems

3.2 Key Structural Changes from A1

4 Concrete Architecture

4.1 Subsystem 1 — CLI Frontend (packages/cli/src/)

4.2 Subsystem 2 — Orchestration Engine (packages/core/src/core/)

4.3 Subsystem 3 — Tool Execution Layer (packages/core/src/tools/)

4.4 Subsystem 4 — Context and Configuration Manager (packages/core/src/config/, packages/core/src/resources/, packages/core/src/core/prompts.ts)

4.5 Subsystem 5 — Safety and Approval Layer (packages/core/src/safety/, packages/core/src/policy/, packages/core/src/confirmation-bus/, packages/core/src/scheduler/)

4.6 Subsystem 6 — MCP Auth Module (packages/core/src/mcp/)

5 Concrete Dependencies

6 Concrete Architecture Diagram - Gemini CLI

7 Architecture Derivation Process

7.1 Stage 1 — Reference the A1 Conceptual Architecture

7.2 Stage 2 — Import Codebase into SciTools Understand

7.3 Stage 3 — Map Source Folders to Subsystem Clusters

7.4 Stage 4 — Identify Divergences and Refine

8 Subsystem Deep Dive

8.1 Conceptual Architecture Breakdown

8.1.1 Subcomponent 1 - Session Manager

8.1.2 Subcomponent 2 - Turn Executor

8.1.3 Subcomponent 3 - API Abstraction

8.1.4 Subcomponent 4 - Tool Execution Gate

8.2 Concrete Architecture Breakdown

8.2.1 Subcomponent 1 - Session Manager (GeminiClient - client.ts)
8.2.2 Subcomponent 2 - Turn Executor (turn.ts)
8.2.3 Subcomponent 3 - API Abstraction (ContentGenerator interface, contentGenerator.ts, geminiChat.ts)
8.2.4 Subcomponent 4 - Tool Execution Gate (coreToolScheduler.ts)
9 Use Cases and Sequence Diagrams
9.1 Use Case 1 - Files and Shell Commands
9.2 Use Case 2 - Web Search and Todos
10 Reflexion Analysis - High Level
10.1 Divergences
10.2 Convergences
11 Reflexion Analysis - Subsystem - Orchestration Engine
11.1 Absences
11.2 Unexpected Components
12 Lessons Learned
13 Conclusion
14 AI Collaboration Report
14.1 Reviewing Rubric Feedback and Providing Feedback
14.2 Planning Document Outline
14.3 Identifying Reflexion git-based Intuition
14.4 AI Collaboration Methodology
16 References

1 Abstract

This report presents a concrete architectural analysis of the Gemini CLI system based on direct examination of its TypeScript source code. The goal of this report was to understand the system's real implementation architecture and compare it with the conceptual architecture. Using SciTools Understand and manual inspection of the repository, the system is decomposed into six primary subsystems:

- CLI Frontend
- Orchestration Engine
- Tool Execution Layer
- Context and Configuration Manager
- Safety and Approval Layer
- MCP Authentication Module

The source folders, classes, and method dependencies are mapped to these subsystems to create a high-level concrete architecture. The orchestration engine manages the conversational lifecycle and coordinates tool execution through a ReAct-style reasoning loop. A subsystem analysis reveals the internal structure behind the session management, message processing, API abstraction, and tool scheduling.

Our reflexion analysis compares the conceptual and concrete architectures, which reveals several structural changes, such as MCP functionality, which is implemented as a specialized tool inside the Tool Execution layer instead of being a standalone subsystem, while the Safety and Approval functionality is implemented as a pipeline, distributed across multiple modules. However, the separation between the CLI interface and core reasoning engine remains consistent with the original conceptual model.

Overall, the analysis demonstrates how examining source code provides a more accurate understanding of system architecture and reveals implementation decisions which are invisible in high-level conceptual designs.

2 Introduction

This report documents how we derived and validated the concrete architecture of Gemini CLI from source code artifacts, then compared it against our A1 conceptual model. The report is organized as a numbered technical narrative: updated conceptual architecture, concrete subsystem decomposition, dependency evidence, derivation workflow, use-case sequences, reflexion analysis, and AI-collaboration audit.

Our main objective was to eliminate the A1 weaknesses by making methodology explicit, tightening traceability from diagrams to code, and ensuring every major claim is supportable through source artifacts and repository history.

2.1 Group 4 Feedback (Improvement Point Applied)

Group 4 feedback emphasized clearer motivation and traceability for architecture claims. We applied this in three concrete ways:

1. We changed the report to a strict numbered structure (1, 2, 2.1, 3, 3.1, ...), so argument flow and ownership are easily viewable.
2. We added explicit derivation staging (A1 baseline -> Understand import -> file-to-subsystem mapping -> divergence refinement) in Section 7.
3. We aligned high-level diagram edges, sequence diagrams, and reflexion rationale to concrete method calls and repository references, along with adding a legend for the diagrams.

3 Updated Conceptual Architecture

The conceptual architecture has been updated to reflect the actual organization of the system as discovered during the concrete analysis of the gemini-cli repository [1]. While our initial model correctly identified the primary functional pillars, the concrete derivation revealed that certain responsibilities—specifically regarding safety, protocol integration, and configuration—are more modular or integrated than originally hypothesized.

This updated model serves as a "revisited" high-level view. It removes specific method calls and implementation details to focus on the six primary subsystems that govern the system's behaviour.

3.1 Revised Subsystems

- **Subsystem 1: CLI Frontend** The entry point for all user interactions, responsible for command parsing and the React-based terminal UI. It maintains a high-level coordination role, initiating sessions via the Orchestration Engine.
- **Subsystem 2: Orchestration Engine:** The core reasoning center of the application. It manages the "Reason and Act" (ReAcT) loop, maintains session state, and coordinates between the user's requests and the available toolset.
- **Subsystem 3: Tool Execution Layer** A unified framework containing all local and remote executable capabilities. This subsystem now conceptually includes the execution logic for the Model Context Protocol (MCP), treating it as a first-class tool type.
- **Subsystem 4: Context and Configuration Manager** A centralized resource handler that assembles system prompts, manages workspace-level settings, and provides the environmental context necessary for the model to function.
- **Subsystem 5: Safety and Approval Layer** A protective pipeline that gates all tool executions. It is responsible for content safety filtering, policy evaluation, and managing the communication bus for user approvals.
- **Subsystem 6: MCP Auth Module** A specialized module dedicated to handling OAuth and authentication providers for external protocol connections, supporting the Tool Execution Layer.

3.2 Key Structural Changes from A1

The primary update to this conceptual model is the redistribution of the **Safety and Approval** and **MCP** components. Originally viewed as monolithic blocks, they are now represented as integrated layers and specialized modules that interact across the core-to-tool boundary. Additionally, the **Context Manager** is refined to show its role in feeding the Orchestration Engine during initial chat setup.

4 Concrete Architecture

The concrete architecture of Gemini CLI was derived from direct analysis of the TypeScript source code using SciTools Understand [7] and manual inspection of the packages/cli/src/ and packages/core/src/ directories [2]. We identified six subsystems, each corresponding to a distinct set of source folders. Their responsibilities and key classes are described below, and all dependency relationships are shown in Figure 1.

4.1 Subsystem 1 — CLI Frontend (packages/cli/src/)

The CLI Frontend handles all user-facing interaction: parsing input commands (commands/), rendering the terminal UI (ui/), managing workspace configuration (config/), and providing service coordination APIs (services/). Concretely, this subsystem also contains event-driven tool execution scheduling hooks in ui/hooks/useToolExecutionScheduler.ts and ui/hooks/useToolScheduler.ts, which synchronize UI state with backend tool execution events.

4.2 Subsystem 2 — Orchestration Engine (packages/core/src/core/)

The Orchestration Engine drives every conversation. Its key concrete classes are GeminiClient (client.ts), GeminiChat (geminiChat.ts), Turn (turn.ts), the ContentGenerator interface (contentGenerator.ts), and CoreToolScheduler (coreToolScheduler.ts). This subsystem manages the full session lifecycle, drives the ReAcT reasoning-and-acting loop [4], abstracts Gemini API communication behind swappable implementations, and gates all outgoing tool execution through the approval pipeline [4].

4.3 Subsystem 3 — Tool Execution Layer (packages/core/src/tools/)

All executable tool implementations are housed here: File tools (read-file.ts, write-file.ts, edit.ts, glob.ts, ls.ts, grep.ts), Shell (shell.ts), Web (web-fetch.ts, web-search.ts), Memory (memoryTool.ts), and Todos (write-todos.ts). MCP tool execution is also housed in this subsystem via mcp-client.ts, mcp-client-manager.ts, and mcp-tool.ts, making MCP a first-class tool type within the tool registry rather than an isolated module. All tool declarations are aggregated by ToolRegistry (tool-registry.ts), the boundary class that the Orchestration Engine queries for available tools.

4.4 Subsystem 4 — Context and Configuration Manager (packages/core/src/config/, packages/core/src/resources/, packages/core/src/core/prompts.ts)

This subsystem assembles the system prompt via getCoreSystemPrompt() in prompts.ts, manages workspace-level configuration through the Config class, and serves file and memory resources used during

sessions. The Orchestration Engine calls into this subsystem during GeminiClient.startChat() to build the instruction context delivered to the model before each session.

4.5 Subsystem 5 — Safety and Approval Layer (packages/core/src/safety/, packages/core/src/policy/, packages/core/src/confirmation-bus/, packages/core/src/scheduler/)

This subsystem is concretely composed of four modules that operate as a pipeline: the safety/ module runs content safety checks (checker-runner.ts, built-in.ts); the policy/ module evaluates tool approval rules via PolicyEngine (policy-engine.ts); the confirmation-bus/ module provides the MessageBus class for routing confirmation events between the core and the CLI UI; and the scheduler/ module orchestrates the full approval pipeline and executes approved tool calls through ToolExecutor (scheduler/tool-executor.ts).

4.6 Subsystem 6 — MCP Auth Module (packages/core/src/mcp/)

This module provides OAuth and authentication providers (auth-provider.ts, google-auth-provider.ts, oauth-provider.ts, oauth-token-storage.ts) for authenticating connections to external MCP servers. The Tool Execution Layer calls into this module when establishing authenticated MCP server connections.

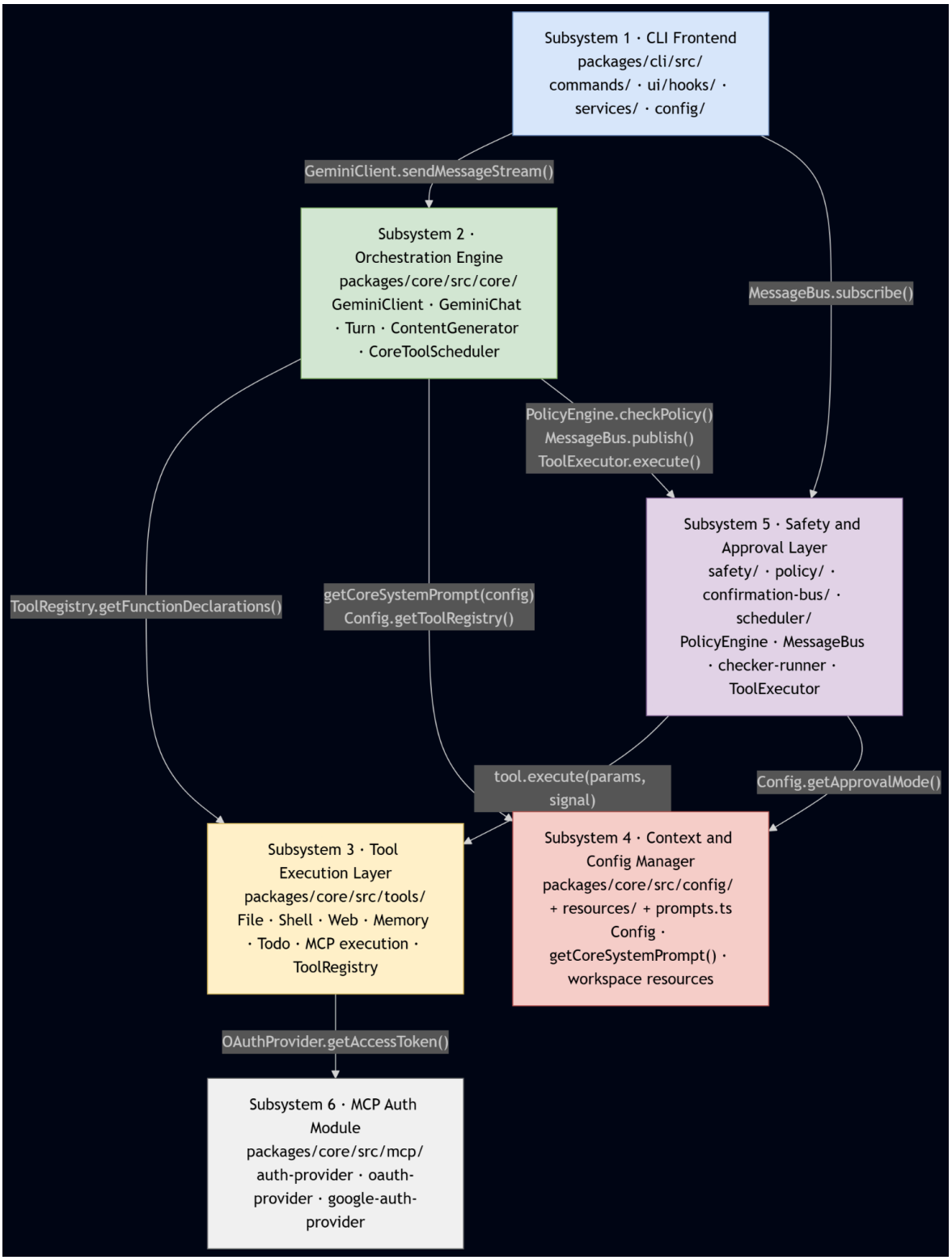
5 Concrete Dependencies

The eight dependency relationships between subsystems, each labelled with the concrete method call that constitutes the dependency, are:

1. **CLI Frontend → Orchestration Engine**: useGeminiStream calls GeminiClient.sendMessageStream() to begin each conversational turn.
2. **CLI Frontend → Safety and Approval Layer**: useToolExecutionScheduler subscribes to MessageBus events to surface tool confirmation prompts to the user.
3. **Orchestration Engine → Tool Execution Layer**: GeminiClient.startChat() calls ToolRegistry.getFunctionDeclarations() to register all available tools in the session.
4. **Orchestration Engine → Context and Configuration Manager**: GeminiClient.startChat() calls getCoreSystemPrompt(config) and Config.getToolRegistry() to assemble session context.
5. **Orchestration Engine → Safety and Approval Layer**: CoreToolScheduler routes each pending tool call through PolicyEngine.checkPolicy(), dispatches confirmation events via MessageBus.publish(), and delegates approved executions to ToolExecutor.execute().
6. **Tool Execution Layer → MCP Auth Module**: mcp-client-manager.ts calls OAuthProvider.getAccessToken() on the providers from mcp/ to obtain credentials for authenticating connections to external MCP servers.
7. **Safety and Approval Layer → Tool Execution Layer**: ToolExecutor.execute() calls tool.execute(params, signal) on the individual tool implementation in tools/ to perform the actual file, shell, web, or MCP operation.

8. **Safety and Approval Layer → Context and Configuration Manager**: `PolicyEngine.checkPolicy()` reads `Config.getApprovalMode()` and the shell command allowlist from the `Config` object to determine whether a tool call requires user confirmation.

6 Concrete Architecture Diagram - Gemini CLI



Symbol	Meaning
Rectangle	Subsystem (label shows folder path)
Labeled arrow (→)	Dependency — label shows the key method call or interaction
Colour	Distinguishes subsystem identity only; has no architectural meaning

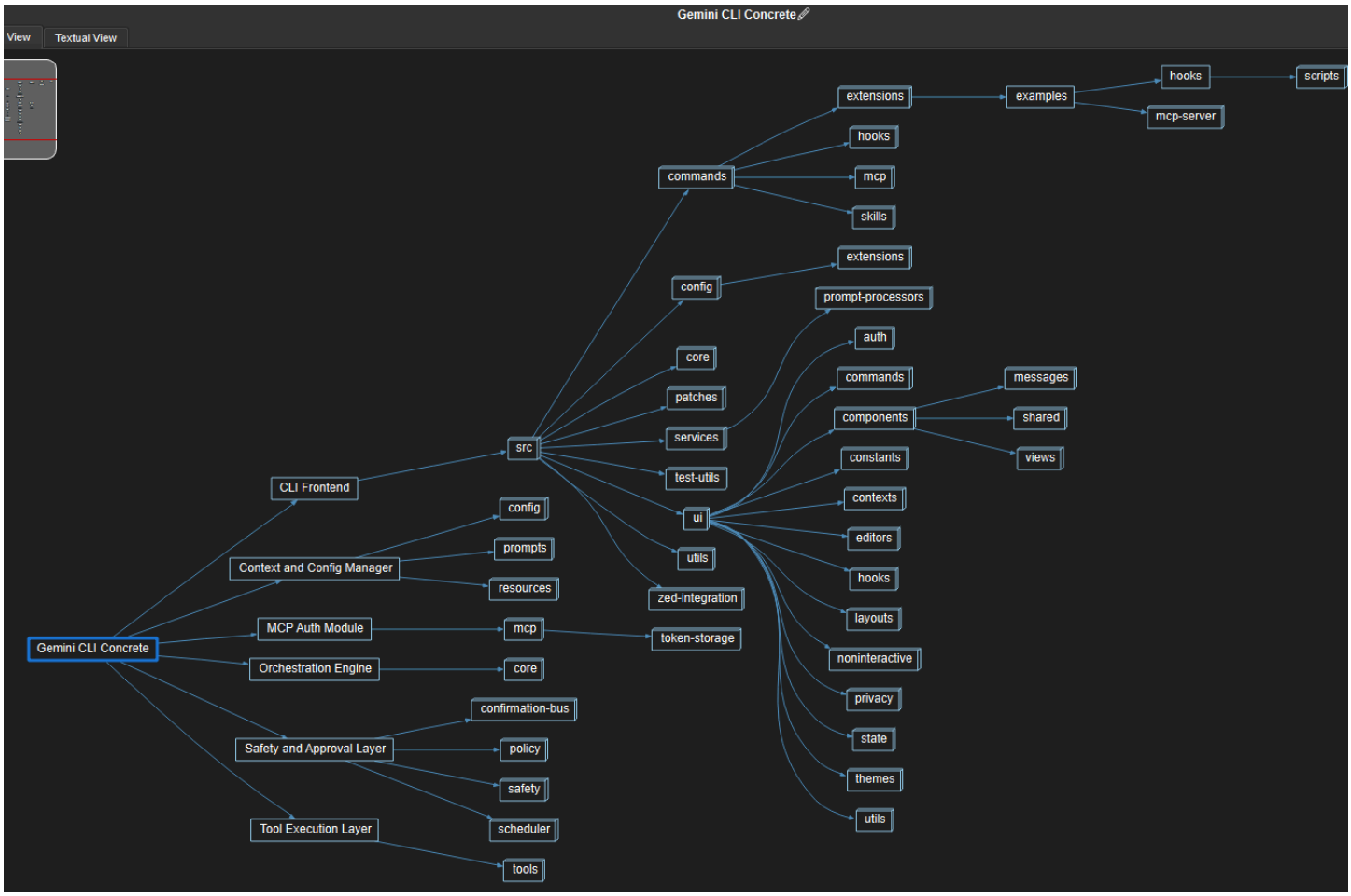


Figure 2A: SciTools Understand Architecture Designer (full mapped architecture view for Gemini CLI)

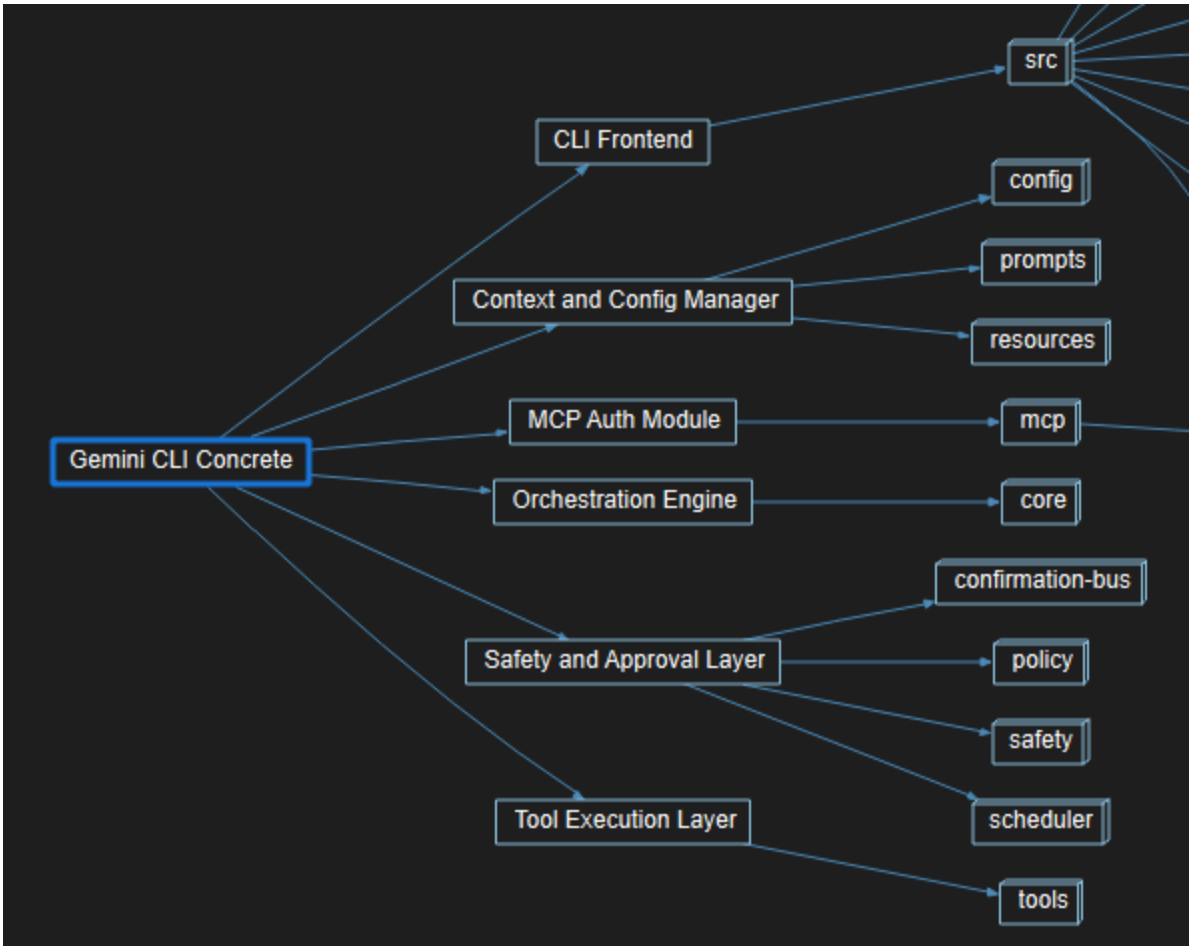


Figure 2B: SciTools Understand Architecture Designer (zoomed view showing the six concrete subsystems used in the A2 architecture derivation).

7 Architecture Derivation Process

The concrete architecture was derived through a four-stage process, beginning with the A1 conceptual model and progressively validating and refining it against the actual codebase [7].

7.1 Stage 1 — Reference the A1 Conceptual Architecture

We began with the six subsystems defined in A1: CLI Frontend, Core Backend, Tool Execution Layer, MCP Integration, Context Manager, and Safety/Approval. These served as the initial cluster definitions against which source files would be mapped.

7.2 Stage 2 — Import Codebase into SciTools Understand

The Gemini CLI TypeScript codebase was imported into SciTools Understand by initializing the project file gemini-v2.und. The project settings (settings.xml) were configured with a TypeScript source filter covering the full packages/ directory, ensuring all CLI and Core source files were indexed for dependency analysis.

7.3 Stage 3 — Map Source Folders to Subsystem Clusters

Using Understand's dependency viewer, source folders were iteratively assigned to the A1-defined subsystem clusters. The initial mapping was:

- packages/cli/src/ → CLI Frontend
- packages/core/src/core/ → Core Backend
- packages/core/src/tools/ → Tool Execution Layer
- packages/core/src/mcp/ → MCP Integration (attempted)
- packages/core/src/safety/, policy/, confirmation-bus/, scheduler/ → Safety/Approval
- packages/core/src/config/, resources/, core/prompts.ts → Context Manager

7.4 Stage 4 — Identify Divergences and Refine

Three divergences between the conceptual model and the concrete source structure were identified during the mapping process:

1. **MCP execution lives inside tools/** — the MCP Integration cluster from A1 could not be confirmed as a separate module. Inspection of the codebase revealed that mcp-client.ts, mcp-client-manager.ts, and mcp-tool.ts are nested inside packages/core/src/tools/, confirming that MCP is implemented as a tool type within the existing registry.
2. **Safety/Approval spans four concrete modules** — rather than mapping to a single folder, the approval pipeline was distributed across safety/, policy/, confirmation-bus/, and scheduler/. This was the result of an explicit refactor performed by the development team [5].
3. **Tool scheduling hooks exist in the CLI Frontend** — useToolExecutionScheduler.ts and useToolScheduler.ts were found in packages/cli/src/ui/hooks/, introducing an unexpected dependency from the UI layer into the tool execution scheduling mechanism.

These divergences are documented in full detail in the Reflexion Analysis sections below. The final concrete architecture reflects the actual module boundaries and dependency directions observed in the source code after this iterative refinement process.

8 Subsystem Deep Dive

For the updated conceptual architecture and new concrete architecture, we have identified that an interesting subsystem to dive deeper into than the diagrams showcase would be the **Orchestration Engine**, as this holds

the core workflow of what makes Gemini CLI useful together. Within this engine, we identified a few subcomponents that are key to the engine's success execution strategy to enable the **Gemini API** and **Service Layer** to function appropriately. First, we will describe them at a conceptual level and then dive deeper into the code to concretely identify how it works.

8.1 Conceptual Architecture Breakdown

8.1.1 Subcomponent 1 - Session Manager

The biggest part of any service-based application is to keep track of the current user session (mostly), along with initializing and storing it appropriately for future sessions. Within Gemini CLI, the session would mostly store conversation data and send the data to the required sources within the source code for processing and actionability, but not send everything; only some sections of the session need to be given to certain parts of the backend. Thus, the Session Manager is also responsible for compressing history when the conversations get too long and cleaning up the session based on the required context for other components. But this subcomponent is only there to update the session based on other actions the backend has taken. How is this done?

8.1.2 Subcomponent 2 - Turn Executor

This subcomponent is the main change-driver for the user session. In our A1 deliverable, we didn't mention the Reason and Act (**ReAcT**) loop that plays into the Orchestration Engine, where **Turns** are the iterative steps within this loop [2]. The ReAcT loop is used by Gemini CLI to solve tasks via breaking down tasks, possible executable actions, and observed results and turning this into a sequence of **Turns**, which are the actions it will take to solve the current problem or task it is facing. As such, the **Turn Executor** will be the component that starts streaming requests, processes events as they arrive into the ReAcT loop, and decides what is necessary to do when events pop up mid-stream/execution of a task in order to allocate the tasks appropriately.

8.1.3 Subcomponent 3 - API Abstraction

We very briefly delved into this topic in A1, but to go more in-depth, this subcomponent is necessary for communication between the user and the Gemini CLI backend to be kept separate and indistinguishable between components. This is accomplished through the use of an abstracted interface, which is implemented at the required component levels, to enable model access in a modular fashion and to ensure security and safety/approval can be done at a modular level easily.

8.1.4 Subcomponent 4 - Tool Execution Gate

Finally, while the **Tool Execution Framework** (**TEF**) is a completely separate subsystem from the **Orchestration Engine**, the TEF still needs to know how to effectively schedule and approve these tools for use without needing to know the nitty-gritty details of what the Engine is running the tool on exactly. As such, this subcomponent is responsible for scheduling tool usage in an effective manner, ensuring tools do not have

parallel calls since Gemini CLI doesn't know the side effects that could happen between tools, and coordinating the approval necessary before executing a tool in the first place.

8.2 Concrete Architecture Breakdown

Now that we have gone over the Conceptual breakdown of the subcomponents within the **Orchestration Engine**, we may now delve into their concrete implementation within the code. The folder that holds all of these subcomponents is `packages/core/src/core`.

8.2.1 Subcomponent 1 - Session Manager (GeminiClient - client.ts)

As the session manager, this is the component the CLI talks to at the top-level before anything gets prepared for execution. There are numerous methods vital to the process outlined in the conceptual architecture. The `startChat()` method helps build the initial history via assembling the context of the user's environment and preparing any initial chat-based context for the model (instructions, such as system prompts and tool lists, that are given to the chatbot to consider before a message is sent in a new chat). Meanwhile, for conversation cleaning and compression, the `tryCompressChat()` function is called when context budget is nearly full, and calls the **Turn Executor's** same function, which is the actual execution pipeline, which snapshots curated history, rebuilds the chat context by summarizing the chat window and builds a new chat via `startChat()` with this new context. Additionally, the `resetChat()` method is responsible for completely regenerating the chat context window to its initial state, when necessary. Finally, a powerful function that is exposed throughout the Gemini CLI API is `generateJson()`, which allows any component to directly bypass certain components and allows for users or Gemini CLI workflows to directly inject utility calls within a message for additional functionality.

8.2.2 Subcomponent 2 - Turn Executor (turn.ts)

The main functionality of this subcomponent begins when the `run()` function is called for a particular instance of a **Turn**, where this function begins the streaming loop for the instance. This will allow the **Turn** instance to pull streaming events from `GeminiChat.sendMessageStream()`, where each event is parsed into a subclass of `GeminiEventType` (e.g. `Content`, `ToolCallRequest`, etc.). When a `ToolCallRequest` event is specifically raised, it calls `CoreToolScheduler.schedule()` so that the tool calls can be appropriately scheduled for use, as mentioned in the Concrete Architecture. Once the tool call is completed, a promised response is fed back into the turn executor stream that was initialized earlier, enabling the turn to continue. Thus, this subcomponent is where the agentic loop formally runs, and the processes are derived here.

8.2.3 Subcomponent 3 - API Abstraction (ContentGenerator interface, contentGenerator.ts, geminiChat.ts)

In order to allow for backend communication to be robust and allow different types of content to be ingested for processing, the `ContentGenerator` interface has every call to the Gemini API pass through it. The

`GeminiChat` object that is currently initialized will hold an instance of `ContentGenerator` and will use it for the message streaming in the **Turn Executor**. As such, this flexibility of the interface allows for the abstraction over various auth methods and API implementations so that in different deployment environments (production, test, development, etc.), there are minimal changes required to the way incoming and outgoing messages are processed at a high level.

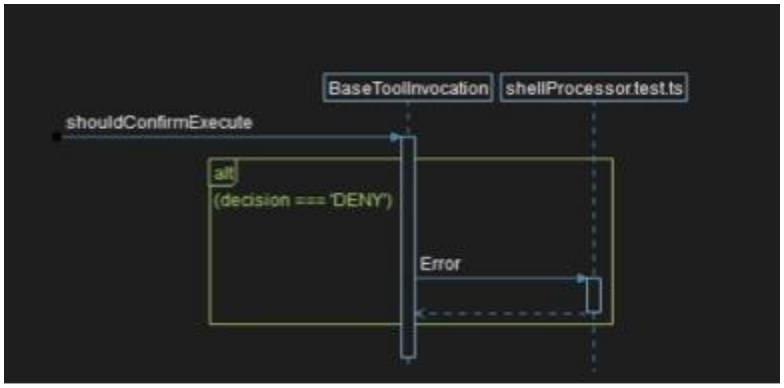
8.2.4 Subcomponent 4 - Tool Execution Gate (coreToolScheduler.ts)

Finally, this subcomponent is where the tool calls scheduled by the **Turn Executor** are actually run. This component enforces the constraint that no new tool calls are scheduled while others are currently executing or awaiting approval, much like how it was mentioned in the concrete architecture above. The requested tool for a particular instance is first looked up via the tool registry, which is part of the **Tool Execution Layer** subsystem, and then `tool.shouldConfirmExecute()` is called to determine if user approval is needed, and then approved tools are finally executed using `tool.execute()`. The result of the execution is then sent back to the **Turn Executor,** as mentioned before, so that the results are fed into the executor stream.

9 Use Cases and Sequence Diagrams

9.1 Use Case 1 - Files and Shell Commands

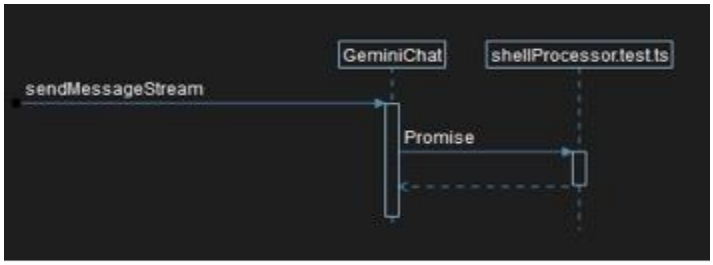
Entry point: shouldConfirmExecute in [coreToolScheduler.ts]



The sequence diagram shows shouldConfirmExecute being called on a BaseToolInvocation instance, representing the Safety and Approval Layer intercepting a pending shell or file tool call. The alt (decision === 'DENY') fragment captures the branching approval logic described in the concrete architecture. If PolicyEngine.checkPolicy() returns a deny decision, an Error is propagated back through shellProcessor.test.ts in the Tool Execution Layer and the tool call is cancelled. If approved, execution proceeds to ToolExecutor.execute(). This diagram concretely illustrates the Orchestration Engine to Safety and Approval Layer to Tool Execution Layer dependency chain documented in the concrete architecture.

9.2 Use Case 2 - Web Search and Todos

Entry point: sendMessageStream in geminiChat.ts



The sequence diagram shows sendMessageStream being called on GeminiChat, which returns a Promise to shellProcessor.test.ts in the Tool Execution Layer. This represents the streaming turn lifecycle described in the Turn Executor subcomponent section. Turn.run() calls GeminiChat.sendMessageStream() and asynchronously processes incoming events. For web search and todo workflows, this stream delivers ToolCallRequest events that CoreToolScheduler picks up to schedule the appropriate web or todo tool. This diagram concretely illustrates the CLI Frontend to Orchestration Engine to Tool Execution Layer dependency chain documented in the concrete architecture.

10 Reflexion Analysis - High Level

The reflexion analysis for the Gemini CLI system involves a systematic comparison between the six subsystems conceptualized in our conceptual (A1) report and the concrete architecture derived from the TypeScript source code. By mapping our initial mental model against the "ground truth" of the repository, we identified several structural divergences, a subsystem absence, and two key convergences. These findings highlight the evolution of the system from a theoretical design to a highly modular, event-driven implementation.

10.1 Divergences

The most significant architectural surprises appeared where functional requirements led to a more distributed implementation than our original hierarchical model suggested.

Divergence 1: Integration of MCP into the Tool Execution Layer. In A1, we hypothesized that the Model Context Protocol (MCP) was a standalone subsystem (Subsystem 4) responsible for external service discovery. However, the concrete architecture reveals that MCP is not an independent module; rather, it is integrated directly into the **Tool Execution Layer**. Files such as `mcp-client.ts` and `mcp-tool.ts` reside within `packages/core/src/tools/`.

- **Git-Based Evidence:** Repository inspection confirmed that the `mcp/` directory is reserved exclusively for OAuth authentication providers, while the actual logic for executing MCP-based tools is handled by the `ToolRegistry`.
- **Interpretation:** The developers opted to treat MCP servers as a specialized tool type. This simplifies the Orchestration Engine's logic by allowing it to invoke local shell commands and remote MCP tools using a unified interface, rather than maintaining two separate discovery pipelines.

Divergence 2: Distribution of the Safety and Approval Layer. Our conceptual model viewed Safety and Approval as a single, centralized module (Subsystem 6). Concretely, this layer is a distributed pipeline spanning four distinct modules: `safety/` (content checking), `policy/` (rule evaluation), `confirmation-bus/` (event transport), and `scheduler/` (orchestration).

- **Git-Based Evidence:** Historical analysis of the codebase indicates a major architectural refactor intended to decouple these concerns. For instance, the introduction of the `MessageBus` was a pivotal change that allowed the system to route "approval-required" events between the core and the UI without a direct hard-coded dependency.
- **Interpretation:** This divergence represents a maturation of the architecture. By decomposing safety into a functional pipeline, the system can independently scale its safety checks (e.g., adding a PII filter) without altering the logic of the tool execution scheduler.

Divergence 3: Downward Dependencies in the CLI Frontend. We initially assumed a strict linear flow where the CLI Frontend was purely a consumer of the Core Backend. However, the concrete architecture shows that tool scheduling logic partially resides in the UI layer through `useToolExecutionScheduler.ts` and `useToolScheduler.ts` within `packages/cli/src/ui/hooks/`.

- **Git-Based Evidence:** The existence of these 200+ line scheduling hooks in the CLI package reveals an "unexpected" downward dependency from the UI into the execution mechanism.
- **Interpretation:** Because the CLI is React-based and event-driven, the frontend requires its own scheduling coordination to synchronize terminal UI states with backend background tasks. This violates our A1 conceptualization of a clean, isolated layer separation.

10.2 Convergences

Despite these discrepancies, the core "pillars" of the system remained remarkably consistent with our initial analysis.

Convergence 1: Tool Execution Layer Structure The concrete `packages/core/src/tools/` directory perfectly mirrors the tool groupings predicted in our A1 report. The File, Shell, Web, and Memory tools are organized into logical clusters, all aggregated by a central `ToolRegistry` boundary class. This confirms our initial intuition regarding how the agent expands its capabilities.

Convergence 2: CLI/Core Package Boundary The high-level separation between the user interface (`packages/cli/`) and the reasoning engine (`packages/core/`) is strictly maintained. The CLI communicates with the engine exclusively through the `GeminiClient` interface (specifically via `sendMessageStream`), validating our conceptual model of the system's primary communication bridge.

11 Reflexion Analysis - Subsystem - Orchestration Engine

Above, the reflexion analysis at a high level introduced the divergences that linked the group missing the **Orchestration Engine** in the A1 architecture, while still having the necessary parts to create it, and then creating it in the A2 concrete architecture. Now, we will dive deeper into the different subcomponents and how they diverge from the A1 conceptual architecture.

11.1 Absences

Our A1 Subsystem 5, the **Context Manager**, has no components at all within the concrete architecture and is thus a redundant subsystem. While there is no explicit GitHub commit, pull request, or issue that can help with this, one can notice that context management happens within a few different portions of the code. Of course, no class called **contextManager.ts** or anything similar exists. However, some brief code analysis and prior work shown in the **Orchestration Engine** section show that context assembly happens in various places, amongst some not mentioned, but namely in `GeminiClient`, `GeminiChat`, `prompts.ts`, and `environmentContext.ts`.

11.2 Unexpected Components

The unexpected components here are regarding the **Session Manager** and **API Abstraction** components.

For the **Session Manager**, the unexpected component was the `tryCompressChat()` functionality, which is vital to enabling further chats to execute when the chat history becomes quite large. The discrepancy can be found in Pull Request #2898, which specifically describes that it will "Reduce the threshold for when we compress history from 95% of the token limit to 70%." [5]. This was done because they "ideally only care about the recent N turns and can summarize some of the less recent tasks/files/tool outputs" [5].

Whereas for the **API Abstraction**, we did not identify that there are two possible paths to reach the actual Gemini chat model from the higher-level components. The obvious one, which is what is identified as a workflow, is the `sendMessageStream()` functionality, which lives within the **Turn** component to process messages and send queries to the chatbot. However, what failed to be realized was that there was a piece of functionality that enabled access directly to Gemini without needing processing by the backend, which is via `generateJson()`. The discrepancy can be found in Issue #5082, which explicitly documents the following in the issue description: "packages/core/src/core/client.ts: (The Bypass Path) Note the generateJson method,

which is the entry point for un-logged utility calls" [3]. The indicator of "The Bypass Path" clearly indicates the function as a way to bypass the typical data flow, in this case, specifically for unlogged calls.

12 Lessons Learned

Throughout the development of this analysis of concrete architecture, we gained a deeper understanding of how theoretical architectural models compare with real-world implementations. By examining the Gemini CLI TypeScript code and mapping it using Understand, we found that software systems often deviate from their original conceptual design. While the high-level architecture from our first report had a good starting point, the concrete analysis revealed several structural divergences and implementation details that weren't visible at the conceptual level.

One key lesson was the importance of validating architectural assumptions against actual source code. During the reflexion analysis, we discovered that certain subsystems we initially believed to be independent were instead combined into existing modules. For example, the MCP functionality was implemented as a specialized tool within the Tool Execution Layer rather than being a standalone subsystem. Similarly, the Safety and Approval subsystem was implemented as a distributed pipeline across multiple modules instead of a single component. These findings showed how real systems often prioritize modular flexibility and maintainability over keeping to the original architectural diagram.

Additionally, we gained experience using architectural analysis tools to extract structural relationships from the codebase. Importing the Gemini CLI repository into Understand allowed us to visualize dependencies between packages and identify subsystem boundaries directly from the code. This process showed how important automated analysis tools are when working with complex software systems. Overall, this assignment strengthened our ability to connect conceptual design with concrete implementation.

13 Conclusion

This report presented the concrete architecture of Gemini CLI through direct analysis of its TypeScript source code. Using Understand, we identified the six major subsystems: CLI Frontend, Orchestration Engine, Tool Execution Layer, Context and Configuration Manager, Safety and Approval Layer, and MCP Auth Module. These components work together to support the system's AI-driven workflows and tool execution capabilities.

The reflexion analysis showed that many aspects of the A1 conceptual architecture were accurate; several structural differences exist in the implemented system. Particularly, the MCP functionality was integrated into the Tool Execution Layer, and the Safety and Approval system works as a distributed pipeline. Overall, the analysis shows how examining the concrete architecture of a system provides insight into how complex software is organized, implemented, and maintained in practice.

14 AI Collaboration Report

We have kept using Claude for the same reasons outlined in A1 (Claude models' superiority in code and architectural understanding compared to other models). However, we further reinforced this notion given this assignment is all about reviewing our architecture and coming up with concrete architecture, along with having to analyze commits, issues, and pull requests very quickly for the reflexion analysis portions. Additionally, Claude released Sonnet 4.6 in between the time of our completion of A1 and the start of A2, which seems to be performing noticeably better than 4.5 was, and we were already quite happy with 4.5's performance. Finally, we already had chat history with Claude during A1 and being able to reference the chats ourselves, along with allowing Sonnet to review the chat history itself for context was a huge plus to help us out with A2, and identifying and fixing issues that stemmed from our work in A1.

14.1 Reviewing Rubric Feedback and Providing Feedback

We prompted Sonnet to take the group feedback we received on our rubric, along with the global feedback presented by Dr. Adams, and the report that we wrote. In return, Sonnet provided insight as to what we missed given all these variables that we confirmed through our own brief looks at all the information and have taken the feedback into account when writing A2.

14.2 Planning Document Outline

Similarly to A1, we gave the project description from the detailed description and the rubric and let it come up with the necessary sections for this report, consolidating lots of information to us that we reviewed and created our own plans for writing. This was given in the same chat that the rubric feedback was discussed in so that Sonnet can have the context of what we could improve on and make notes of it while creating the outline.

14.3 Identifying Reflexion git-based Intuition

For the Reflexion intuition, it would have been immensely difficult for our group to find relevant git-based information to discuss the difference in conceptual architecture from A1 to the concrete architecture in A2. Through a few carefully curated prompts, Sonnet was able to look through the Gemini CLI repository and find relevant information for us based on the respective Reflexion section it was looking at and our identified divergences.

14.4 AI Collaboration Methodology

This time, the use of Sonnet was mainly used as a group during the group meetings to discuss and identify a course of action that would best suit our group given our timelines. Given the timing of the feedback, we initially started with the document outline to get a head start on how and what should be planned out, much like the process in A1. We deliberated on the plan and came up with our own changes that we felt like they'd better suit the overall plan. The plan was quite consistently created given Sonnet also took in the context from our A1 report to generate the new format. After the global feedback was provided, and then the individual group feedback, we understood better how to shape the report overall, especially since Sonnet could also take the feedback and point out discrepancies where we lost marks and what we can do to improve for next time. An

example would be the improper positioning of the Use Cases in A1, which we placed before the conceptual architecture diagram. As such, we adjusted for them to be placed after the concrete architecture diagram for A2.

As for the Reflexion points, this was more of an individual use-case for those who were writing the Reflexion analyses. The main issue with Sonnet was that it hallucinated a few times when attempting to find appropriate git-based information and provided git commits that were not at all relevant to the section. For example, when trying to find git information for the absence of a Context Manager, Sonnet responded with: "The Leslie Liu article (pinned to commit dc0e0b4) shows that context assembly…" while commit dc0e0b4 is about a scheduled release workflow update instead [6]. Thus, careful checks and reading of supplied information was crucial to properly adding context via git info.

We estimate our AI teammate did less work in total than in A1, relying less on it in an effort to produce work that is more consistent with the rubrics and feedback provided. This enables more human-centric checks regarding the submission and better understanding of what is done. While we were understanding what was provided to us before, there were clearly a few gaps that we missed but did overall well in, hence us experimenting by attempting to rely less on it for research and writing than in A1. As such, we give it about 10-15% of the overall effort this time for the group.

Sonnet still remained a capable teammate and was paramount to the team's efficacy in planning out ideas, structure, and what to do better this time. By relying a bit less on Sonnet this time, and also taking our process of prompting the AI teammate from A1 and ameliorating it, we also had less checks and changes that were needed to be made against its responses. For any checks that we had to make, they were relatively straightforward and simple to confirm. We are quite happy with how it was used more effectively this time, and we hope our results can show for it through this adjustment.

15 References

[1] google-gemini, "GitHub - google-gemini/gemini-cli: An open-source AI agent that brings the power of Gemini directly into your terminal.," *GitHub*, Mar. 12, 2026. https://github.com/google-gemini/gemini-cli

[2] Google for Developers, "Gemini CLI," *Google for Developers*, 2026. https://developers.google.com/gemini-code-assist/docs/gemini-cli

[3] abhipatel12, "Bug: /stats under-reports Model API Calls," *GitHub*, Jul. 29, 2025. https://github.com/google-gemini/gemini-cli/issues/5082

[4] imadraude, "Fix inconsistency between schedulers in Plan mode," *GitHub*, Jan. 24, 2026. https://github.com/google-gemini/gemini-cli/issues/17444

[5] Vachan-Shetty, "Reduce the threshold for when we compress history," *GitHub*, Jul. 01, 2025. https://github.com/google-gemini/gemini-cli/pull/2898

[6] Anthropic, "Claude," *Claude.ai*, 2025. https://claude.ai/ (accessed Mar. 10, 2026).

[7] SciTools, "Understand Documentation," *SciTools Docs*, 2026. https://docs.scitools.com/